REQUIREMENTS Elicitation and MODELLING TECHNIQUES

COMP1787 Teaching Week 6

Dr. Aditi Rawal

UNIVERSITY OF GREENWICH

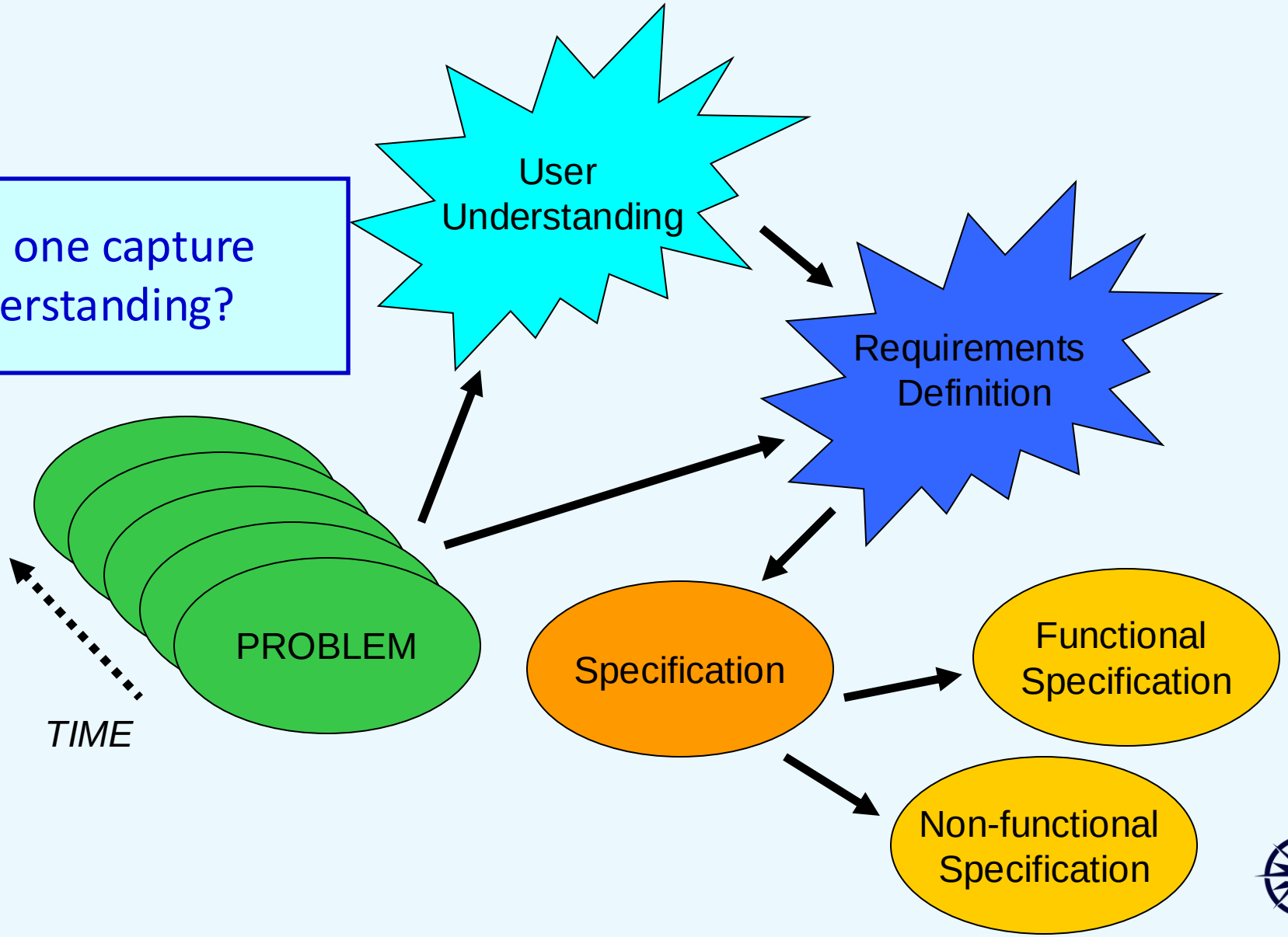
Requirements recap

- Business roles identify who is responsible for defining, agreeing and prioritising requirements
- Facilitated Workshops are used to define and gain buy-in to requirements and to aid the resolution of conflicting requirements
- The Prioritised Requirements List (PRL) is used to document agreement and assist in the management of the requirements throughout the project
- MoSCoW Prioritisation is used to allow consistent, agreed levels of priority
- Requirements evolve – as the project progresses, they become more detailed

UNIVERSITY OF GREENWICH

User Requirements

How does one capture user understanding?



UNIVERSITY OF GREENWICH

Project Requirements

Requirements Engineering – Specifying the Problem (Designing a Solution)

- Understanding requirements can be broken down into:
  - Requirements gathering (**elicitation**)
    - Techniques for research and fact-finding
      - Literature review
      - Quantitative research: Surveys, experimental
      - Qualitative research: Narrative, surveys
  - Requirements **analysis**
    - Understanding functional and non-functional requirements
  - Requirements **specification**
    - documenting the results of elicitation and analysis

UNIVERSITY OF GREENWICH

Requirements Definition

- You are building a **solution** to a problem, seeking to answer a question
- Your **research and data gathering** should help you understand the problem well enough to **define** it and define a procedure to evaluate the merit of solutions
- Work out what is needed to solve the problem as the set of **requirements** that your product must meet

UNIVERSITY OF GREENWICH

Functional and Non-functional Requirements

- Distinction between functional and non-functional requirements.

Example:

1. "The system shall ensure that data is protected from unauthorised access by authentication by password".

Non-functional requirement as it does not specify specific system functionality which must be provided.

2. "Users must be able to login".

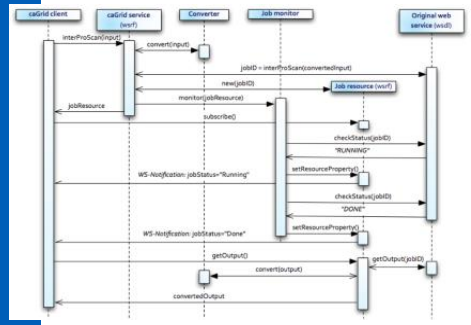
Functional requirement as it specifies a function (user login) which must be incorporated in the system.

UNIVERSITY OF GREENWICH

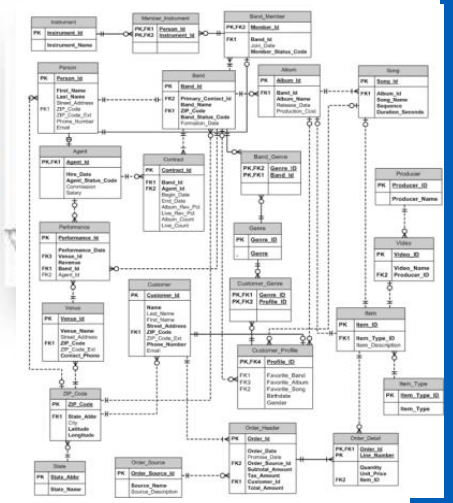
Kotonya, G., Sommerville, I., Non-functional Requirements, www.comp.lancs.ac.uk/computing/resources/re/slides/Chapter8.ppt

Viewpoint Determines Design

Behaviour

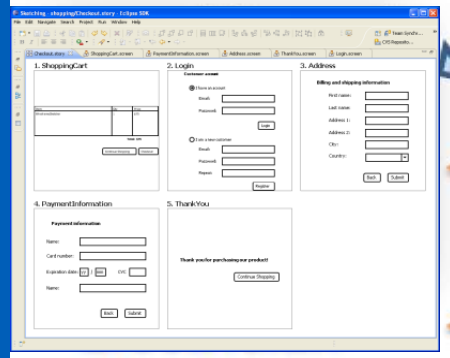


Information Structure

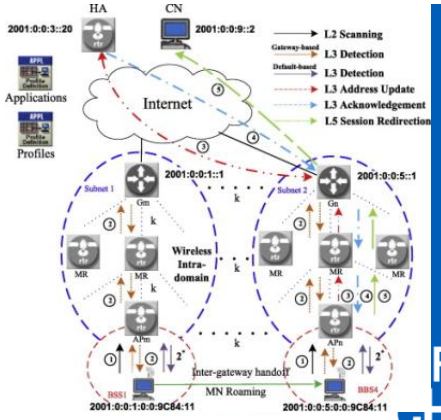


Viewpoints

User Interaction



Network Infrastructure



View

Stakeholder

Web Page Designers

(Has Requirements)

Network Engineers

Orbus Software (2012), *TOGAF 9 In*

- Therefore when tackling client expectations, you must focus on one particular set of requirements (problem) from a viewpoint of interest [*usability, security, performance, functionality,* …]
- Your design should use the tools and techniques appropriate to address that problem.
- The evaluation should be appropriate to testing those requirements

UNIVERSITY OF GREENWICH

Zachman Enterprise Architecture Ontology

	DATA *What*	FUNCTION *How*	NETWORK *Where*	PEOPLE *Who*	TIME *When*	MOTIVATION *Why*	
SCOPE (CONTEXTUAL) *Planner*	List of Things Important to the Business ENTITY = Class of Business Thing	List of Processes the Business Performs Function = Class of Business Process	List of Locations in which the Business Operates Node = Major Business Location	List of Organizations Important to the Business People = Major Organizations	List of Events Significant to the Business Time = Major Business Event	List of Business Goals/Strat Ends/Means=Major Bus. Goal/ Critical Success Factor	SCOPE (CONTEXTUAL) *Planner*
ENTERPRISE MODEL (CONCEPTUAL) *Owner*	e.g. Semantic Model Ent = Business Entity Reln = Business Relationship	e.g. Business Process Model Proc = Business Process I/O = Business Resources	e.g. Business Logistics System Node = Business Location Link = Business Linkage	e.g. Work Flow Model People = Organization Unit Work = Work Product	e.g. Master Schedule Time = Business Event Cycle = Business Cycle	e.g. Business Plan End = Business Objective Means = Business Strategy	ENTERPRISE MODEL (CONCEPTUAL) *Owner*
SYSTEM MODEL (LOGICAL) *Designer*	e.g. Logical Data Model Ent = Data Entity Reln = Data Relationship	e.g. Application Architecture Proc = Application Function I/O = User Views	e.g. Distributed System Architecture Node = I/S Function (Processor, Storage, etc) Link = Line Characteristics	e.g. Human Interface Architecture People = Role Work = Deliverable	e.g. Processing Structure Time = System Event Cycle = Processing Cycle	e.g., Business Rule Model End = Structural Assertion Means = Action Assertion	SYSTEM MODEL (LOGICAL) *Designer*
TECHNOLOGY MODEL (PHYSICAL) *Builder*	e.g. Physical Data Model Ent = Segment/Table/etc. Reln = Pointer/Key/etc.	e.g. System Design Proc = Computer Function I/O = Data Elements/Sets	e.g. Technology Architecture Node = Hardware/System Software Link = Line Specifications	e.g. Presentation Architecture People = User Work = Screen Format	e.g. Control Structure Time = Execute Cycle = Component Cycle	e.g. Rule Design End = Condition Means = Action	TECHNOLOGY MODEL (PHYSICAL) *Builder*
DETAILED REPRESEN-TATIONS (OUT-OF-CONTEXT) *Sub-Contractor*	e.g. Data Definition Ent = Field Reln = Address	e.g. Program Proc = Language Stmt I/O = Control Block	e.g. Network Architecture Node = Addresses Link = Protocols	e.g. Security Architecture People = Identity Work = Job	e.g. Timing Definition Time = Interrupt Cycle = Machine Cycle	e.g. Rule Specification End = Sub-condition Means = Step	DETAILED REPRESEN-TATIONS (OUT-OF CONTEXT) *Sub-Contractor*
FUNCTIONING ENTERPRISE	e.g. DATA	e.g. FUNCTION	e.g. NETWORK	e.g. ORGANIZATION	e.g. SCHEDULE	e.g. STRATEGY	FUNCTIONING ENTERPRISE

© 2008 John A. Zachman, Zachman International, Inc.

The Zachman Enterprise Architecture Ontology

Provides a structured framework for organizing and understanding complex enterprises.

1. **What**: Describes the data and information that make up the enterprise from a business perspective.
2. **How**: Focuses on the processes and activities involved in transforming data into useful information.
3. **Where**: Specifies the locations or physical components where the enterprise operates.
4. **Who**: Identifies the stakeholders and roles within the enterprise, including both internal and external entities.
5. **When**: Addresses the timing and sequencing of events or processes within the enterprise.
6. **Why**: Explores the motivations, goals, and objectives driving the enterprise's activities.

UNIVERSITY OF GREENWICH

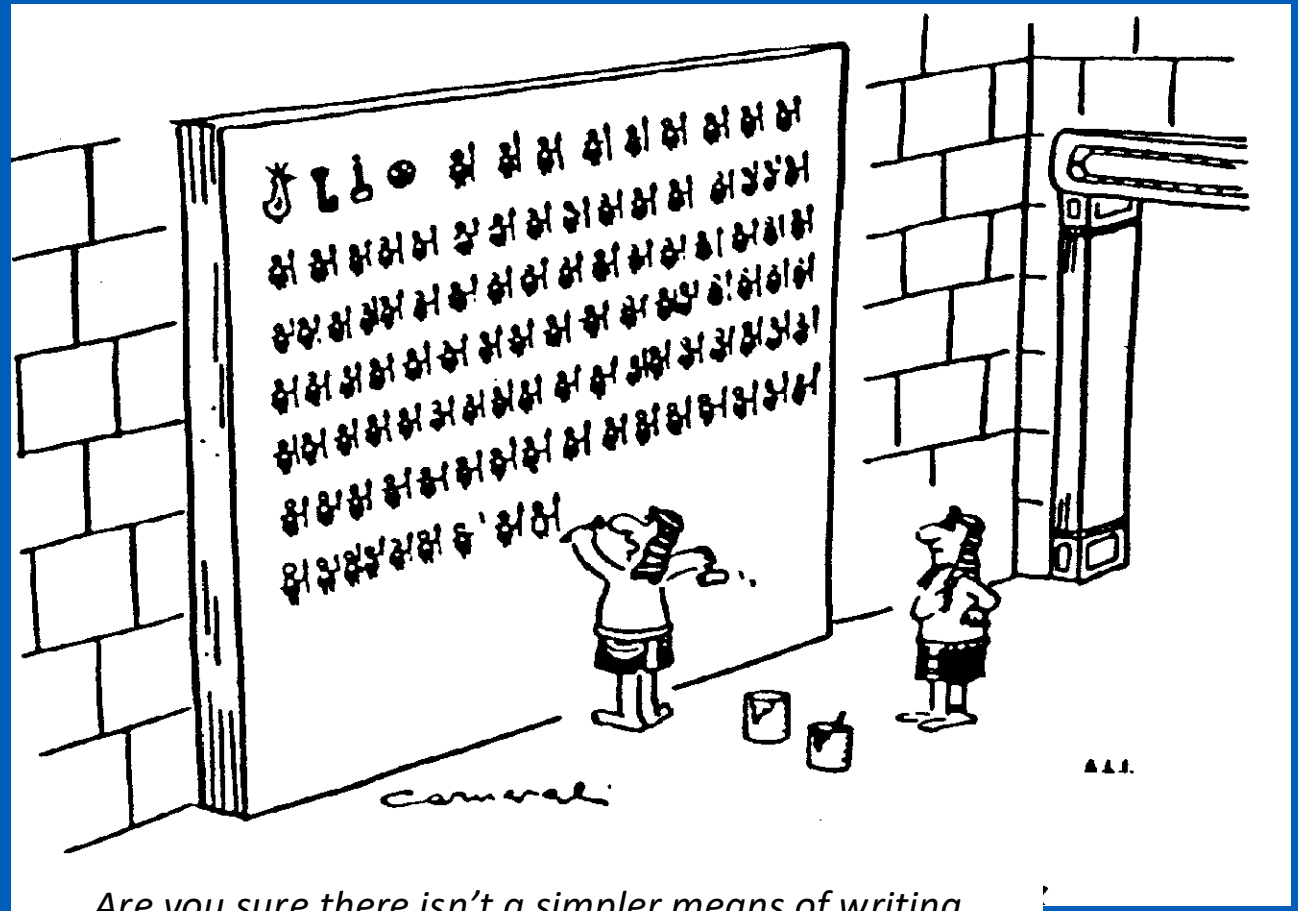
Example

	Planner (Scope - Why do we need this?)	Owner (Business - How does it work?)	Designer (System - What should it include?)	Builder (Technology - How to build it?)	Programmer (Implementation - How to develop it?)	User (Functioning - How to use it?)
What? (Data)	Student records, courses, grades, fees	Data model for students, courses, payments	Entity-Relationship Diagram (ERD), tables for student, course, finance	SQL database, cloud storage	Database schema, indexing	Student records are retrievable and modifiable
How? (Function)	Student registration, fee payment, grading	Business processes for admissions, finance, and academics	UML diagrams, system workflows	Backend services (Java, Python, .NET)	API development, microservices	Students register, pay fees, and view grades
Where? (Network)	Accessible online and on-campus	Portal for students, faculty, admin	Logical architecture (servers, cloud, data centers)	AWS, Azure, on-premises servers	Network security, load balancing	Web/mobile app for student access
Who? (People)	University administration, faculty, students	Roles: Students, Professors, Admins	Access control, authentication	Role-based authentication (OAuth, SSO)	User authentication module	Secure logins for users
When? (Time)	Semester-based enrollment cycles	Admission deadlines, course schedules	Automated notifications & scheduling	Cron jobs, background services	Scheduled tasks for reminders	Students get automated reminders for payments and exams
Why? (Motivation)	Improve student experience, streamline operations	Compliance with university policies	Business rules for enrollment, payments	Rule-based access control	Implementing business logic	Smooth, user-friendly student management

Importance of Notation and Modelling Languages for Expressing Requirements and Design

"By relieving the brain of all unnecessary work, a good notation sets it free to concentrate on more advanced problems, and in effect increases the mental power"

Whitehead A.N., *An Introduction to Mathematics*, OUP, 1959



Are you sure there isn't a simpler means of writing 'The Pharaoh had 10,000 soldiers'?

UNIVERSITY OF GREENWICH

Modelling Notations (Techniques)

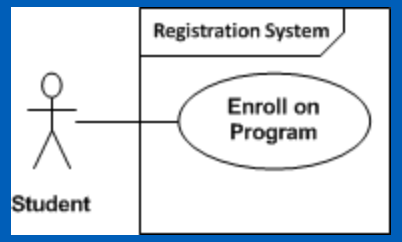
Previous example refined models from concept to code but depending on nature of client project other notations may be more appropriate. *E.g:*

- Describing context – Rich pictures, context diagrams….
- Conceptual modelling of business processes and workflow with BPM
- ERDs for logical and implementation data modelling
- Flowcharts, activity diagrams for describing dynamic behaviour
- Pseudocode to outline algorithms. Formal specification for describing functionality
- Circuit and network architecture diagrams for modelling computer systems
- Modelling enterprise architecture
- Storyboards, website maps and navigation diagrams….

Key question - Does the notation support traceability from the requirements conceptual model, through logical design, to implementation and back again?

UNIVERSITY OF GREENWICH

Expressing Requirements and Design with Models



Contextual model: Present overview. Identify scope of the system, boundaries and interactions

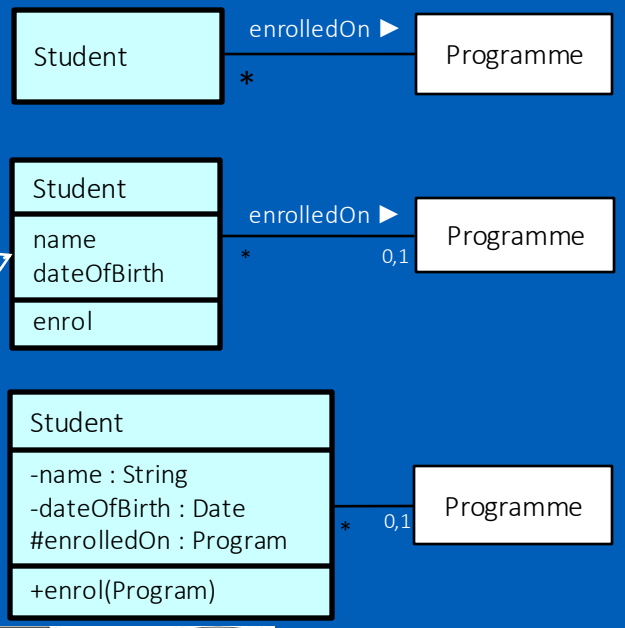
Conceptual computation independent model identifying the basic elements in the system and how they are linked

Logical platform independent model

Platform specific **implementation model**

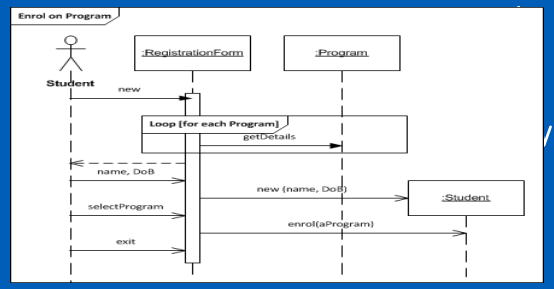
Deployment model

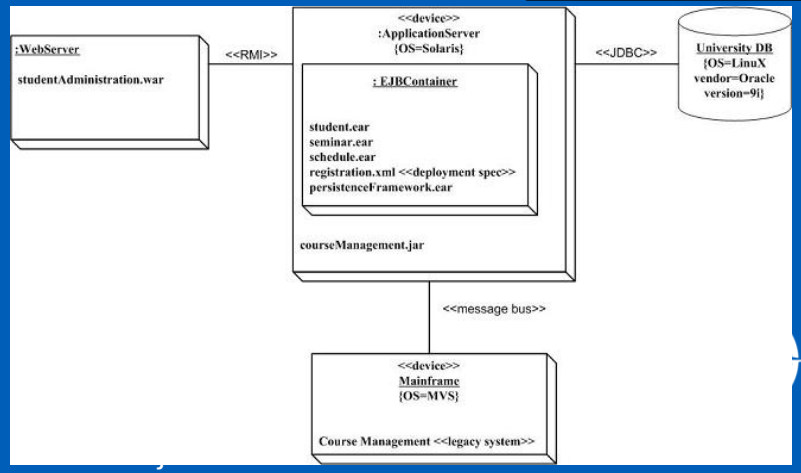
Physical **Implementation**



Static information

Dynamic behaviour

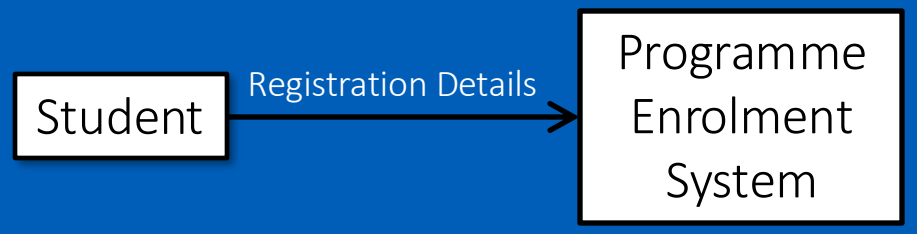


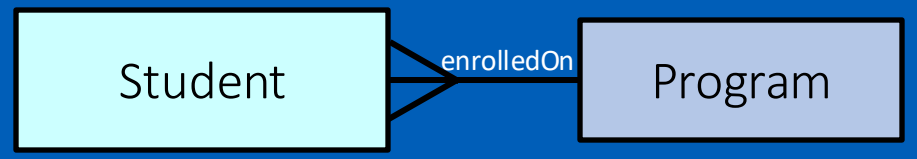


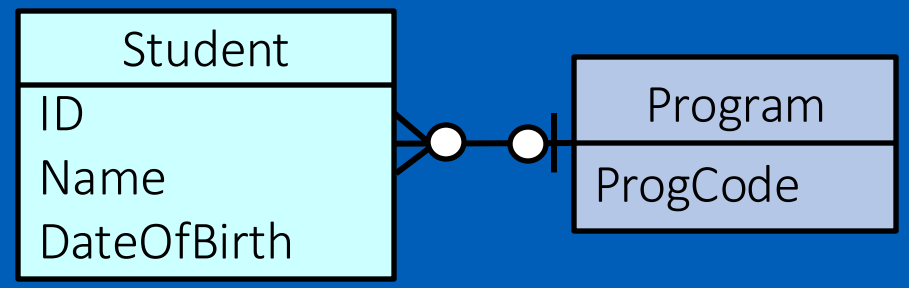
UNIVERSITY OF GREENWICH

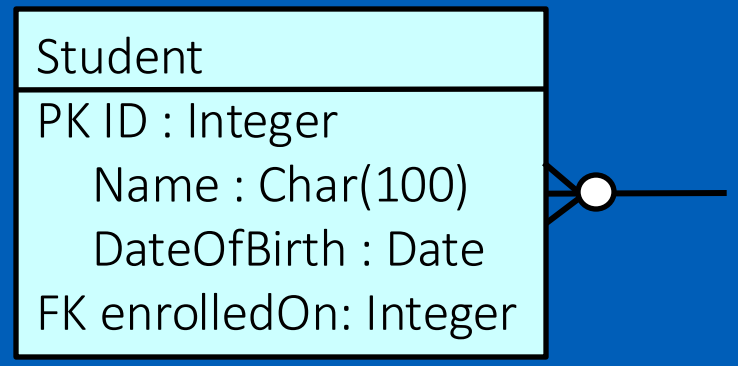
Previous Example Using A Modelling Approach

- **Contextual Model:** Says what the system is for identifying external sources and maps the system processes with a context diagram
- **Analysis:** High level abstract **conceptual model** identifying the basic concepts in the system and the relationships between them
- Refine and add detail to produce a platform independent **logical model** of what data the system needs to hold
- **Design Specification:** Refine into an **implementation model** by adding implementation specific information
- **Implementation:** Translate the model into physical code










```
CREATE TABLE Student (
    ID INT NOT NULL PRIMARY KEY,
    name NVARCHAR(100) NOT NULL,
    dateOfBirth DATETIME,
    enrolledOn INT FOREIGN KEY REFERENCES Program(ProgCode)
);
```

UNIVERSITY OF GREENWICH

Formal Specification

- A requirements definition will often be in plain English
  - may be ambiguous
- Turn it into **formal statements** e.g.
  - requirement lists
  - user stories
  - use cases
- Decide on acceptance tests for each requirement

UNIVERSITY OF GREENWICH

User Story

Story ID: Story Title:

User Story:

As a: <role>

I want: <some goal>

So that: <some reason>

Importance:

Estimate:

Acceptance Criteria

And I know I am done when:

Type:

- ☐ Search
- ☐ Workflow
- ☐ Manage Data
- ☐ Payment
- ☐ Report/ View

- Who (user role) – Is this a customer, employee, admin, etc.?
- What (goal) – What functionality must be achieved/developed?
- Why (reason) – Why does user want to accomplish this goal?

UNIVERSITY OF GREENWICH

- Registered User can add books to the cart

User Story Example 1:
As a Registered Website User,
I want to be able to add books to the cart,
So that I can then proceed to buy listed books from the website.

Acceptance Criteria

1. User can add a book into the shopping basket with correct quantity and price information presented in the cart page.
2. There should be one line for each item in the basket.
3. Each line should show the item (with product id, name and a clickable link to its product page) and its cost.
4. The total cost of all books in the basket should be shown.
5. User is not allowed to enter invalid value as quantity (e.g. negative value, non-numerical value).
6. The number of books allowed to add into the basket must not be greater than the number of books remain in stock.

UNIVERSITY OF GREENWICH

In Class Activity

- Registered User can choose a guaranteed delivery date.

User Story Example 2:
As a Registered Website User,
I want to

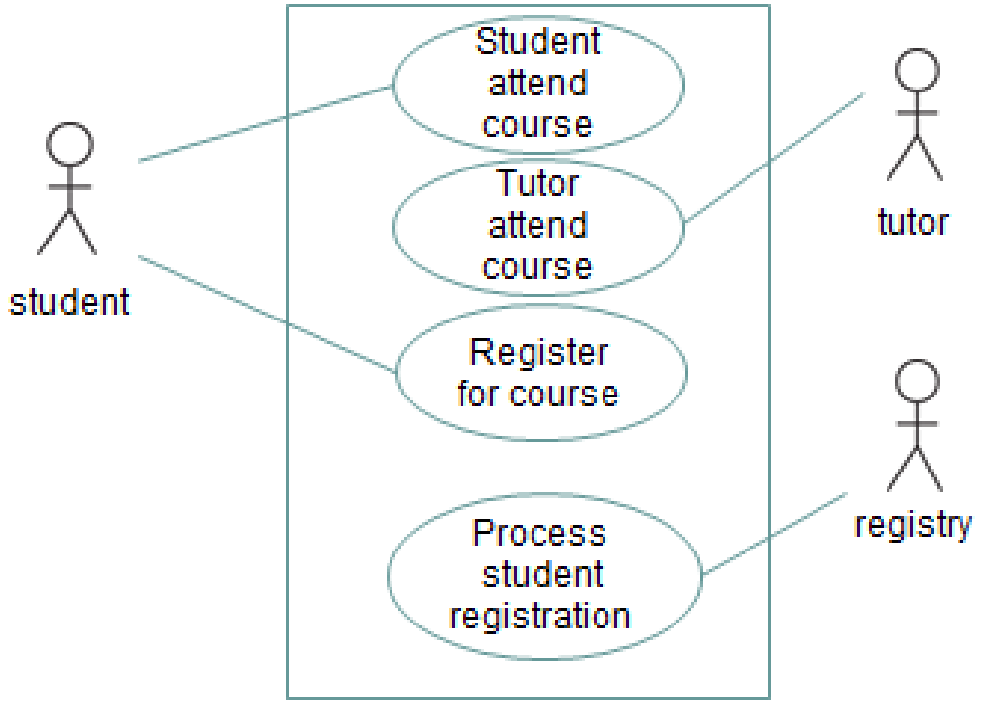
User Story Example 2:
As a Registered Website User,
I want to be able to choose a guaranteed delivery date when I place my order, So that I know when to expect my order

Acceptance criteria:
1. The delivery date will always be the next working day.
2. The delivery date will be on a Monday if the order is placed on a Saturday. 3. The delivery date cut-off time for orders will be 3pm.
4. There will be email confirmation of the delivery date.
5. All product lines have the same delivery date rules.

UNIVERSITY OF GREENWICH

Use case Modelling

From our previous example



UNIVERSITY OF GREENWICH

What goes on inside this Use Case?

Use Case Name: **Register for Course**

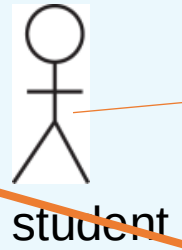
Actor: **Student**

Pre-condition: Student not registered for a course

Whilst completing registration, on click Help button go to help system

1. Student logs on to system
2. **Check Login credentials**
3. Student selects appropriate course to register for
4. Student supplies correct qualification information
5. Student provides correct payment details
6. Student receives confirmation of registration

Post-Condition: Student receives confirmation of course registration process



Register for course

It might be used for other uses of the system

Anyone logging onto system needs to do this

UNIVERSITY OF GREENWICH

<<Includes>> and <<extends>> connections

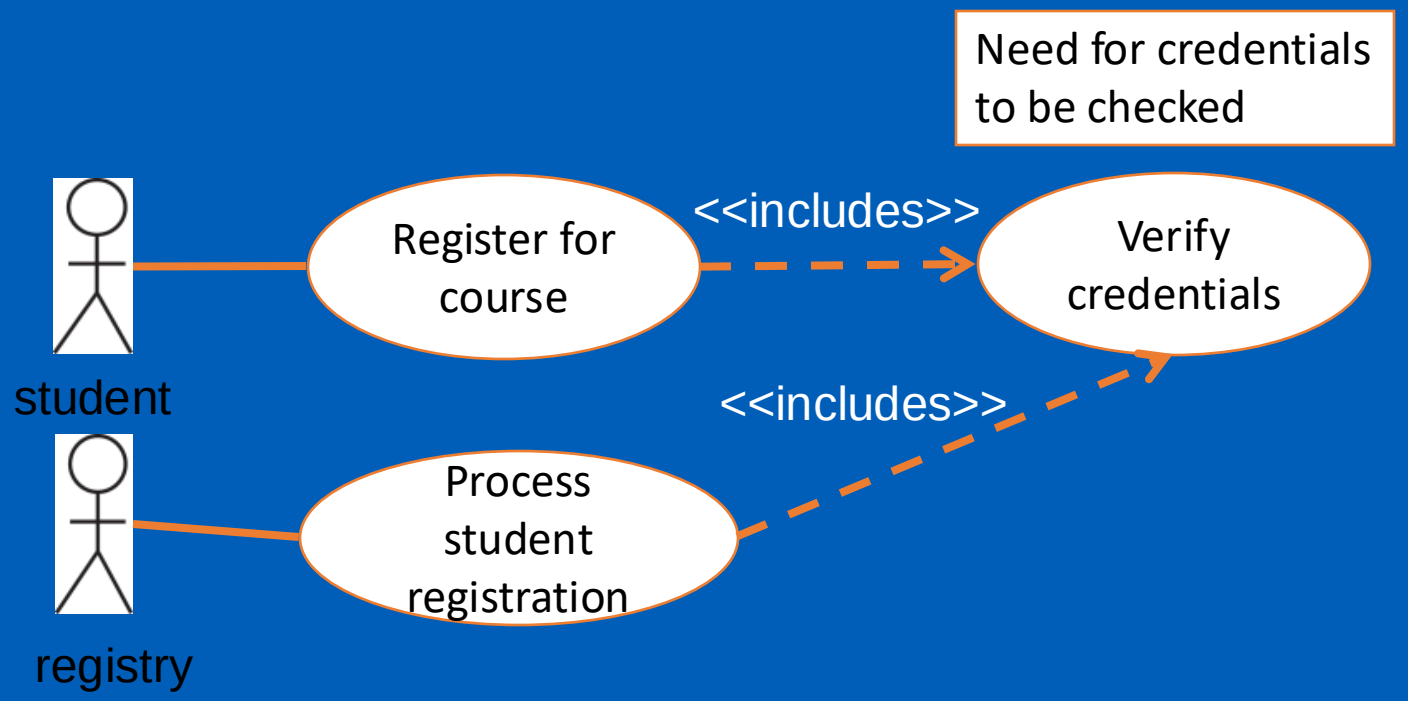
Using the <<includes>> and the <<extends>> connectors

Note:

Going to the help system is optional	Having your credentials validated is mandatory

UNIVERSITY OF GREENWICH

<<includes>>

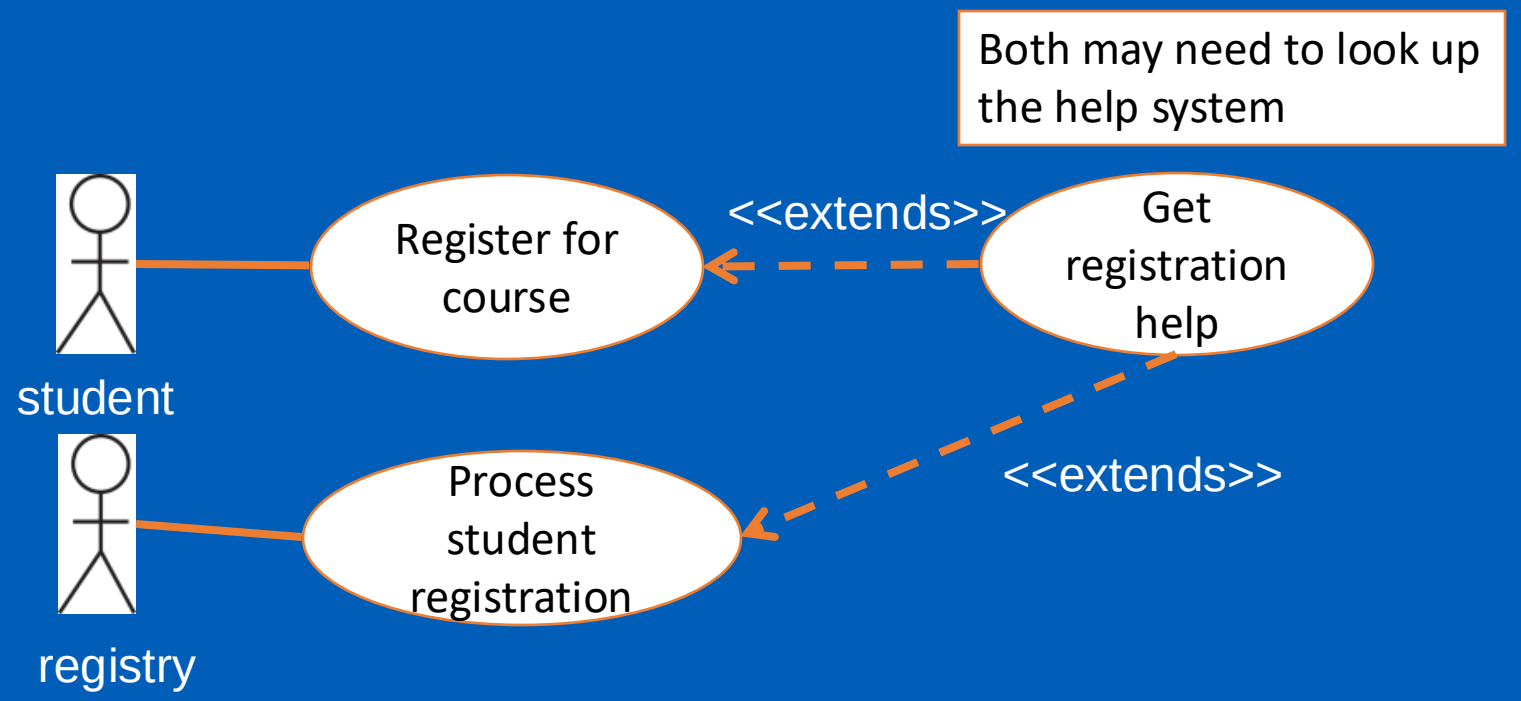


- This solves mandatory repeating use of a Use Case
- It identifies a direct relationship between two use cases

UNIVERSITY OF GREENWICH

<<extends>>

- In this case we need to deal with a relationship between two use cases which is optional



UNIVERSITY OF GREENWICH

<<includes>> and <<extends>>

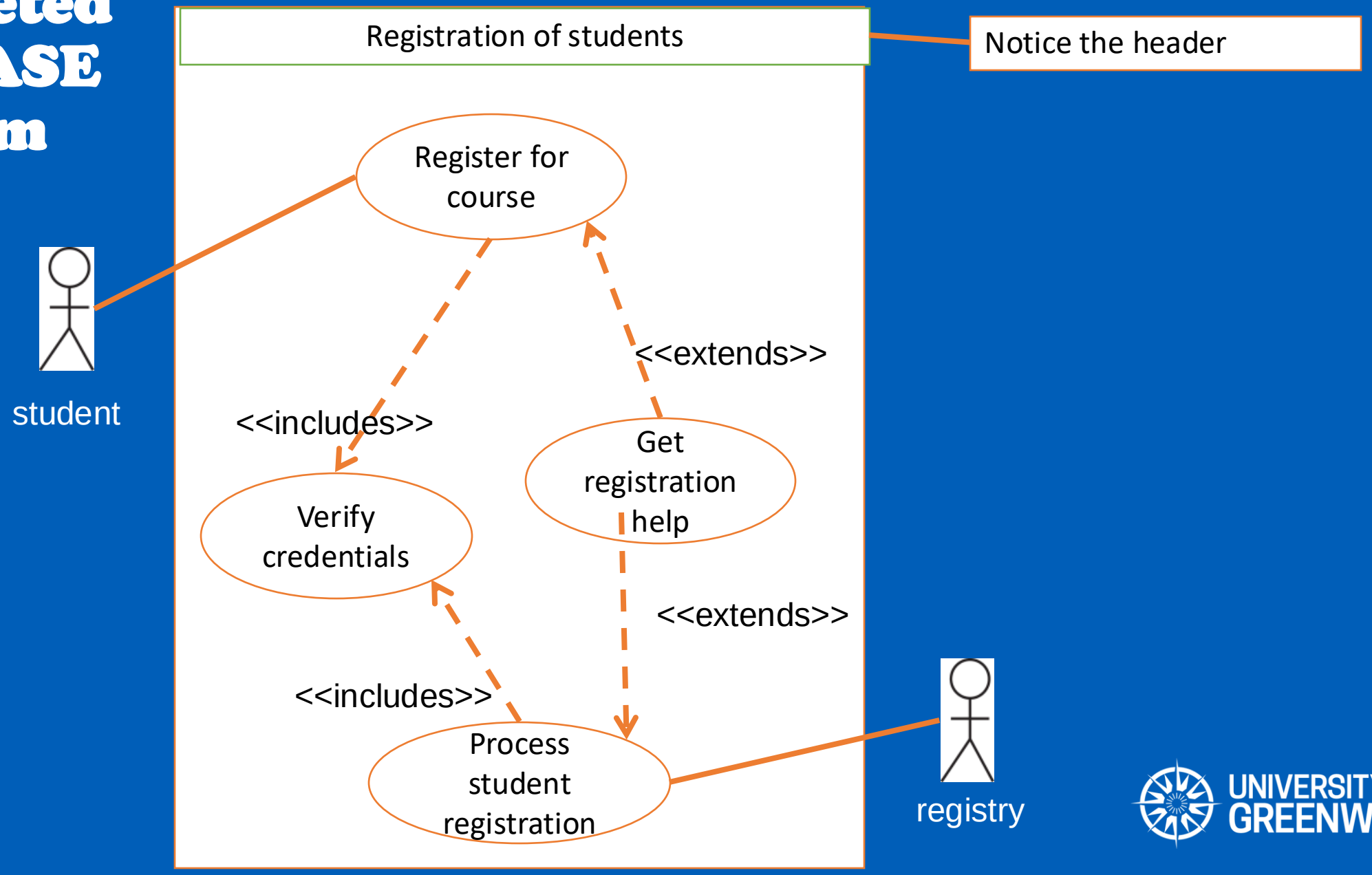
Used whenever you have a use of the system which you want to repeat many times

<<includes>> is a mandatory use

<<extends>> is an optional use

UNIVERSITY OF GREENWICH

Completed USE CASE Diagram



UNIVERSITY OF GREENWICH

Some general guidelines about Use Case diagrams

Think modular

- Break down the problem (or the solution that you have found to the problem) into increasingly smaller parts

Actors are identified by the role (not the individual person)

Start with the actor to identify different uses of the system

Each use case process indicates a complete use of the system

Each Use Case must have a primary scenario and possibly several secondary scenarios attached

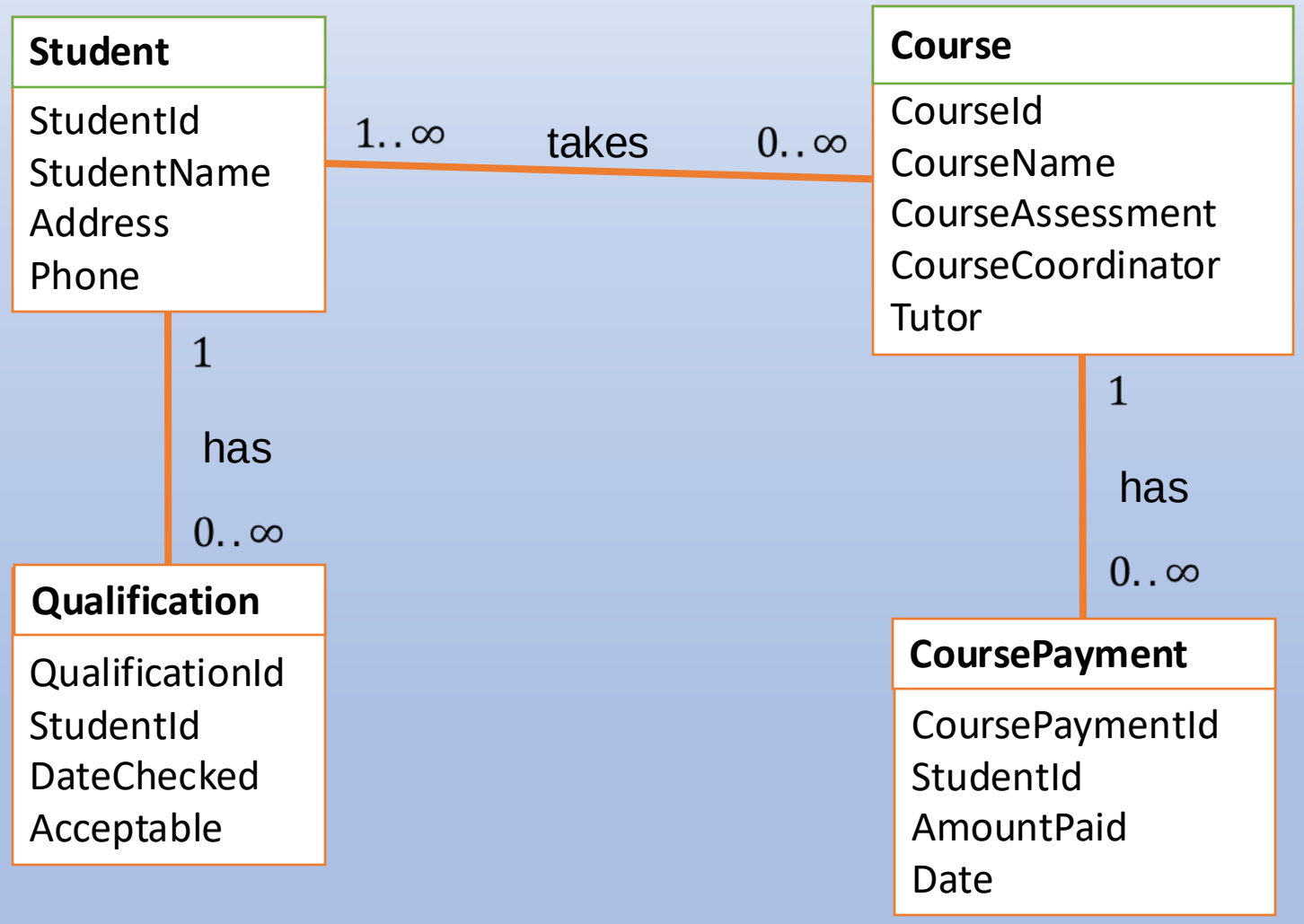
Refine your requirements analysis until your description meets the needs of the organisation

Use case Scenario fo pet adoptio

Use case name	Choose an animal	
Actors	Customer	
Description	This describes how a customer can select a particular animal from the website	
Typical course of events	**Actor Action** **Step 1:**This use case is initiated by a customer **Step 3:** Customer provides username/password **Step 5:** Customer selects an animal from the list **Step 7:** Customer receives confirmation	**System response** **Step 2:** Invoke <<includes>> use case Authorise User. **Step 4:** If correct, Customer is presented with a list of animals **Step 6:** Invoke <<includes>> use case check availability.
Alternate courses	Step 2: If the customer is not already registered, then Invoke <<extends>> use case, Register. Step 4: If username/password incorrect, then inform the customer and start the process again Step 6: If the customer chooses an animal that is not available, then a notification is sent to the customer	
Precondition	This use case is initiated by customers	
Post condition	confirmation	

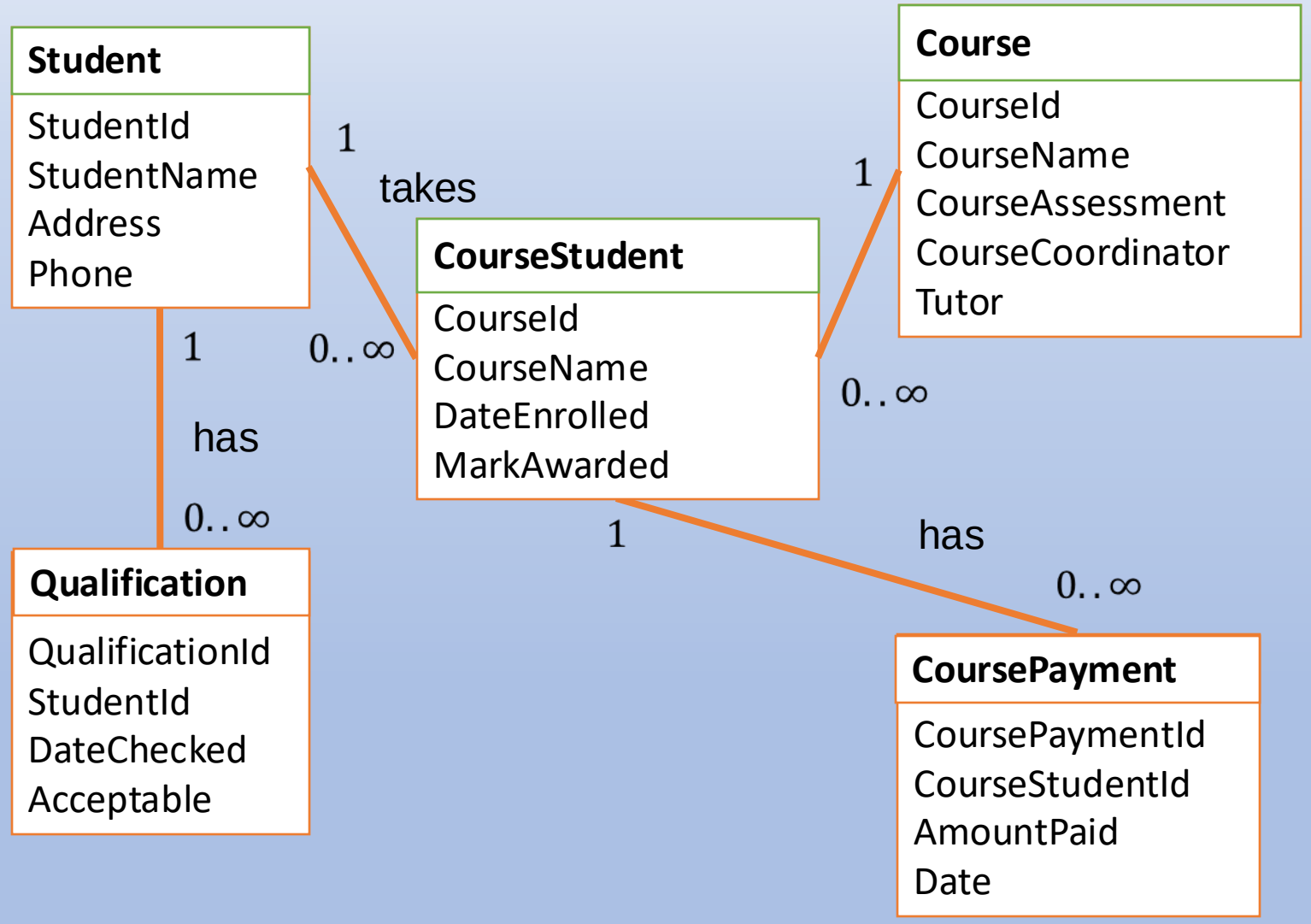
UNIVERSITY OF GREENWICH

Which allows us to start to build a class diagram



UNIVERSITY OF GREENWICH

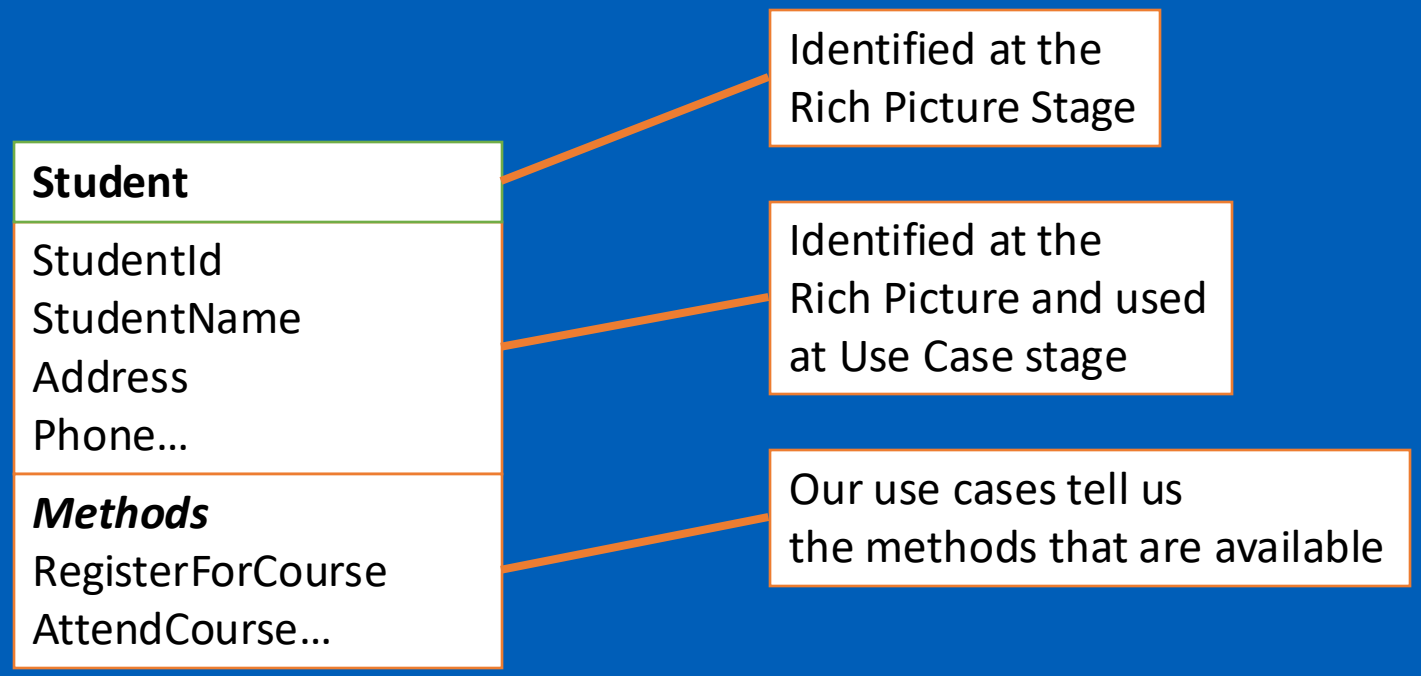
A better model...



UNIVERSITY OF GREENWICH

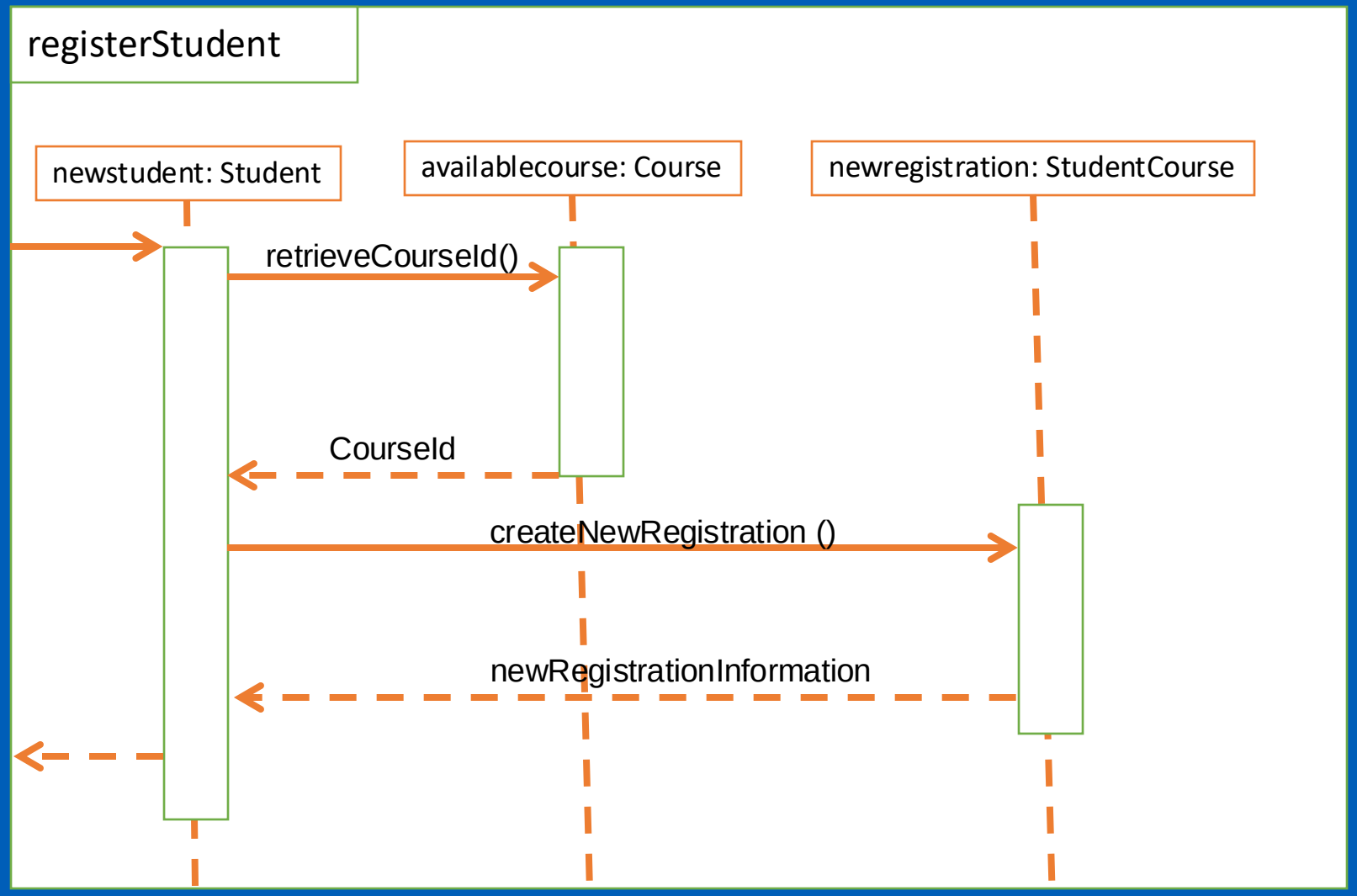
Adding in the methods

- So Use Cases allow us to identify data and methods



UNIVERSITY OF GREENWICH

Then you can produce sequence diagrams…



UNIVERSITY OF GREENWICH

Capturing Design

- Module descriptions
  - e.g. flowcharts, pseudo-code, rich pictures
- Functional design artefacts from UML
  - e.g. use cases, sequence diagrams
- OO design artefacts from UML
  - e.g. class diagrams
- Relational Database design artefacts
  - e.g. data dictionaries, ERD’s, DFD’s
- UI design
  - e.g. screen layouts, navigation, story boards

UNIVERSITY OF GREENWICH

There may be many solutions to any given problem

UNIVERSITY OF GREENWICH

Evaluating against Requirements

How do you know what the problem is and therefore the requirements of a solution?

- What you think the problem is *or*
- Issues identified by review of similar products?
- Issues raised by real world users – What is the evidence? Structured interviews, task analysis?
- Usability issues identified and discussed in the literature? What literature – quality of sources?

How do you collect evidence to evaluate your solution?

- Is this objectively reproducible?
- Comparison with similar products – how? – Is number of bells and whistles a measure of usability?
- Real world users - usability testing (e.g. counting clicks per tasks?), quantitative/ qualitative questionnaires? What is a reasonable range of representative users – where do they come from and are the number of subjects statistically significant?
- How have solutions been previously evaluated? Reproduce/replicate experimental design?

UNIVERSITY OF GREENWICH

Evaluation - Similar Products

If similar products exist, then you must list all of them
or at least most of them

Apply criteria to reduce the list to manageable numbers
what sort of criteria?

Perform detailed evaluation of short-listed products

UNIVERSITY OF GREENWICH

Product Comparison table

Features

Products

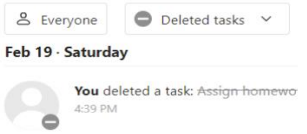
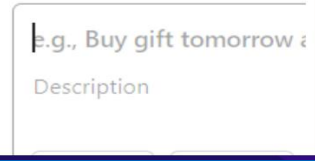
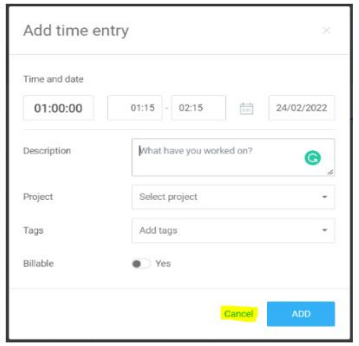
	1	2	3	4	5	6	7	8	9
A	✓		✓		High	-	✓	3	120
B	✓	✓			Low	✓	×	0	144
C	✓	✓	✓	✓	Med	✓	-	3	240
D	✓		✓		Med	-	×	5	64
E		✓	✓	✓	High	✓	✓	5	12
F	✓				Low	-	-	1	100
G	✓	✓	✓		Low	✓	×	9	4

Lacking clarity

Are these features qualitative or quantitative?

UNIVERSITY OF GREENWICH

Example of a good product comparison table

Jakob Nielsen's 10 Usability Heuristics Principles	XYZ		ABC	
	Rating	Notes	Rating	Notes
		ensure understanding of information. General Language English		
User control and freedom	4	• There is no option for user to revert to the previous action, they still can find the deleted tasks in the activity log but can't recover them.  • Users can cancel an action if they click on the button accidentally.  	3	• Users cannot undo the action or recover the deleted task. They also can't find the task once it was deleted. • Users can cancel an action if they click on the calendar accidentally. 

UNIVERSITY OF GREENWICH

How to Evaluate?

- Choose relevant criteria to judge products
- Features
  - e.g. Nielsen's Heuristics, W3C, Scheiderman
- Quantitative
  - e.g. performance, throughput, scalability, popularity
- Qualitative
  - usability, utility, accessibility, navigation, style
- Your criteria choices demonstrate your ability

- Verification
  - are we building the product right?
- Validation
  - are we building the right product?

- Sommerville

UNIVERSITY OF GREENWICH

Some Key Things for Requirements Management

- Identify and express a problem – generalise and abstract
- Identify the criteria that could be used to objectively evaluate how the solution addresses the problem
- Analysis - explore alternatives and potential solutions
- Specify the requirements for a proposed solution in such a way that it can be evaluated as addressing the problem
- Design – provide a broad brush model of a potential solution
- Build, test, run product. Collect and present results of runs
- Using the results collected from the product runs, evaluate proposed solution(s) against identified criteria

UNIVERSITY OF GREENWICH

In summary

- We have considered use case modelling and scenario development at the early stages of the life cycle
  - Now we understand how they can help us clarify requirements, define scope and realise what we don't know.
- We have looked at how use case models and scenarios evolve over time
- We have examined how Use Case Modelling fits together with other models we may use

UNIVERSITY OF GREENWICH

Conclusion

- Clarity of requirements is needed before you can sensibly commence on your product design
- Requirements identify and specify key details
  - functional and non-functional
- Requirements are the parameters against which you can **test** your product
- We have considered use case modelling and how it helps us clarify requirements, define scope and realise what we don't know.
- Use case models and scenarios evolve over time
- We have examined how Use Case Modelling fits together with other models we may use
- Evaluation tests your requirements

UNIVERSITY OF GREENWICH

Questions

UNIVERSITY OF GREENWICH

Further Reading

- Creswell, J.W., *Research Design*, Sage Publications, 2014
- Wright, C., *8 Tips to Capture Better Requirements for your Software Project,* 2011
- Cornella-Dorda *et al*., *A Process for COTS Software Product Evaluation Software Evaluation*, 2004

UNIVERSITY OF GREENWICH